EE 474 Lab Part 2: First SRV02 Operations and Sensor Calibration (1 week)

Contents

1 Introduction **2**
- 1.1 Building and running the Simulink model with HIL blocks . . . 3
- 1.2 Quarc . . . 3
- 1.3 Modeling and calibrating sensors . . . 4
  - 1.3.1 Characteristics of the sensors . . . 4
    - 1.3.1.1 Encoder . . . 4
    - 1.3.1.2 Potentiometer . . . 6
    - 1.3.1.3 Tachometer . . . 7
  - 1.3.2 Models for the sensors . . . 8
    - 1.3.2.1 Encoder . . . 8
    - 1.3.2.2 Potentiometer . . . 9
    - 1.3.2.3 Tachometer . . . 9
- 1.4 Prelab questions . . . 11

2 In Lab **12**
- 2.1 Using Quarc, Quanser's hardware in the loop tools . . . 12
  - 2.1.1 Steps to build and run: . . . 13
  - 2.1.2 HIL tools and an example Simulink model . . . 13
- 2.2 Cabling the SRV02 . . . 14
- 2.3 Exploring sensor characteristics . . . 16
- 2.4 Calibrating the sensors . . . 17
- 2.5 Consider the write-up . . . 18
- 2.6 Consider the principal values . . . 18

3 Write-up **19**
- 3.1 Principal Values . . . 19
- 3.2 Questions . . . 19

4 Appendices **22**
- 4.1 Simulink solver setup . . . 22
- 4.2 Scope setup for logging to Matlab workspace . . . 22
- 4.3 Matlab routine `DisplayStatistics.m` . . . 23

1 Introduction

In this lab you will connect the Windows PC and Simulink to the SRV02 hardware, and work with and calibrate the sensors. Labs for the rest of the semester will include operations with the SRV02 and depend on your sensor calibration results.

A basic simulink model that connects with the SRV02 hardware is illustrated in figure 1. 'HIL' stands for Hardware In the Loop. The HIL blocks in figure 1 provide input and output connections through the Q2-USB to the hardware (see figure 3 of "EE 474 Lab Introduction" to see all elements, including the Q2-USB). Unlike basic Simulink, which doesn't connect to the outside world (no real-world voltages, currents or actions are produced) when a Simulink model includes the HIL blocks, signals coming into or flowing out of those blocks are voltages that are sent to or that come from real-world devices.

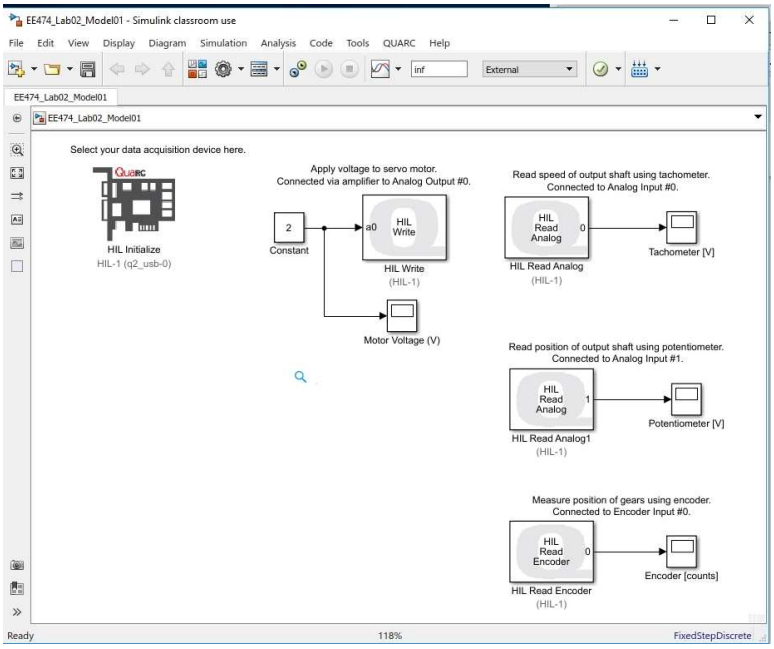


Figure 1: Simulink model `EE474_Lab02_Model01.mdl`. This model provides a constant motor voltage, through the "HIL Write" block, and reads each of the three SRV02 sensors. With proper setup, the scopes record data and transfer it to the Matlab workspace when the simulation is terminated.

Sensors are a critical element of any feedback control system.An ideal sensor would produce an electrical signal always perfectly reflecting the physical quantity being sensed. But few sensors are ideal. Practical sensors have imperfections of various kinds, including scale factors and offsets that must be accounted for, limited range, rollover, and noise. Many challenges in practical control systems, including the SRV02, arise with the real-world characteristics of sensors.

In Lab Part 2, you will investigate and learn about the characteristics of the SRV02 sensors (two position sensors and a velocity sensor). This is often an early step in practical control system development. Sensors are often an expensive component of completed systems, and the trade-off between sensor robustness, measurement range, imperfections and cost is often an important part of a controller development project.

1.1 Building and running the Simulink model with HIL blocks

In Simulink, the ***build*** process translates the simulink model into a C program that implements the simulation model. The user does not generally review or modify this program[1]. For our system, the C program thus built runs with a Mathworks technology called Real-Time Windows Target (RTWT). RTWT runs the simulation directly on the host computer CPU chip, without (or "below" one might say) Microsoft Windows. See figure 2. Using a technique called ***interrupt driven processing***, heavy Real-Time Windows Target processing can slow down Windows processing, but Windows applications can not slow down Real-Time Windows Target processing. On a 64-bit Intel CPU, Real-time Windows Target can achieve an update rate of 5000+ samples per second.

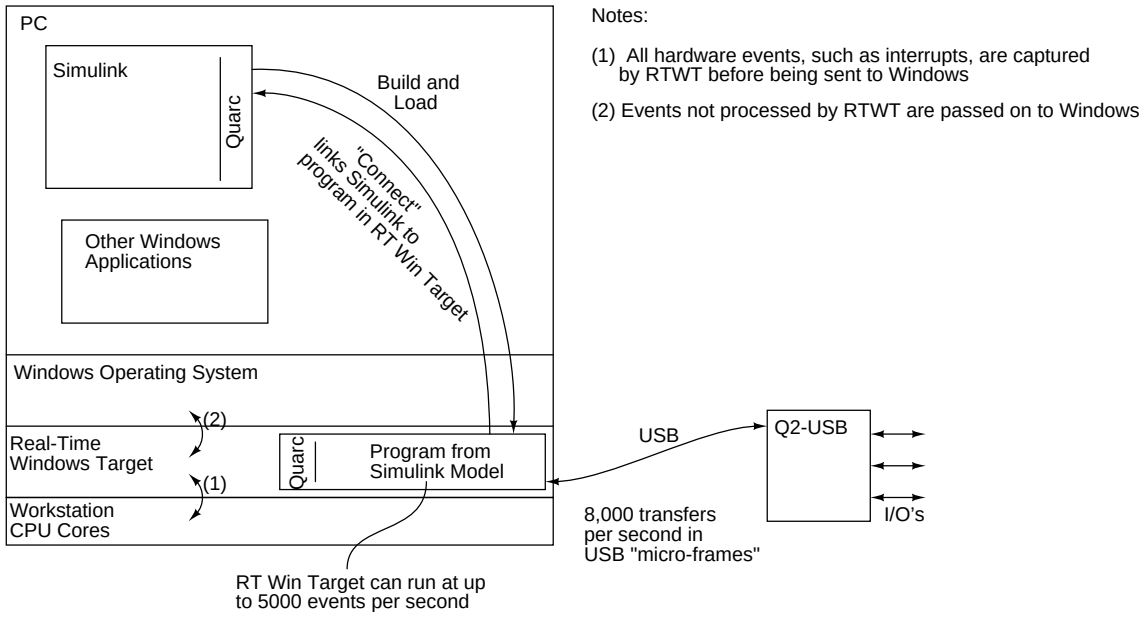


Figure 2: Block diagram showing combination of software elements. The Quarc Simulink libary implements communication between Simulink, the Real-time Windows Target and the Q2-USB.

To load the simulation into the Real-time Windows Target for processing, select the "Connect to Target" button at the top of the model window.

1.2 Quarc

Quarc is a set of software tools developed by Quanser to connect the Simulink model with the real-time component in RTWT with a hardware interface unit, such as Quanser's Q2-USB. Quanser Quarc with the Q2-USB hardware can achieve an update rate of 2000 samples per second.

1. The Mathworks translation tool is called coder and is used with many industrial controls as well as the SpaceX Falcon 9 rocket. To use these steps in critical applications, the translator must have a very high level of reliability!

1.3 Modeling and calibrating sensors

The SRV02 is equipped with three sensors:

- Position sensors:
  1. Shaft encoder
  2. Potentiometer
- Velocity sensor:
  3. Tachometer, built in to the Motor/Tachometer/reduction gear unit.

As with any sensor, there are three elements to using the sensor data: 1) an awareness of the characteristics and limitations of the sensor, 2) a model (that is, an equation) relating the sensed signal (e.g., position) to the output signal (e.g., voltage), and 3) calibration data to accurately calibrate the sensor.

Example data from the SRV02 sensors is seen in figure 3. These data were collected with +1.8 volts applied and the motor turning, using the Simulink model of figure 1. Using the right-hand rule, the machine rotated in the positive direction at a bit more than one revolution per second.

1.3.1 Characteristics of the sensors

The essential characteristics of the SRV02 sensors are:

1.3.1.1 Encoder Shaft encoders are wheels with clear and dark bands, and an optical pickup to detect the alternating bands, as seen in figure 4. The electronic components (in the Q2-USB) count the generated pulses.

Advantages: An advantage of encoders is that the pulses per revolution is inherently calibrated by the number of light and dark bands around the ring. They can also have very high resolution. The resolution of the SRV02 output shaft encoder is a medium-level 4096 counts per revolution.

Disadvantages: Disadvantages are that :

1. Encoders show rollover,
2. They are not continuously reading (there is no way to detect a movement smaller than that corresponding to one pulse), and
3. Many encoders are ***incremental***, the encoder counter counts pulses corresponding to movement, it does not detect the absolute position.

Note: in spite of this list of disadvantages, encoders are widely used in industry; the advantage of intrinsic calibration outweighs many disadvantages.

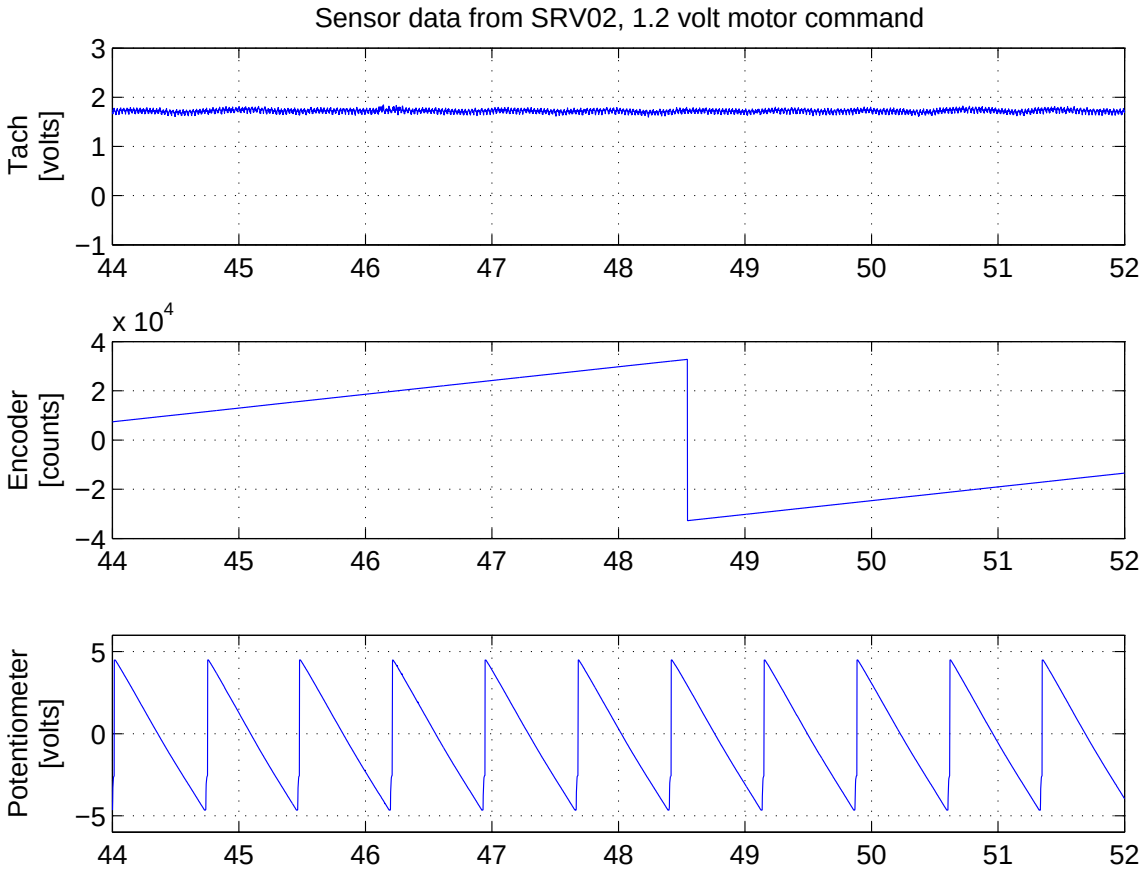


Figure 3: Example plots captured with +1.8 volt applied motor voltage. Rollover is seen in the encoder count at t=48.5 seconds. Multiple rollover events (one per revolution) are seen in the potentiometer output.

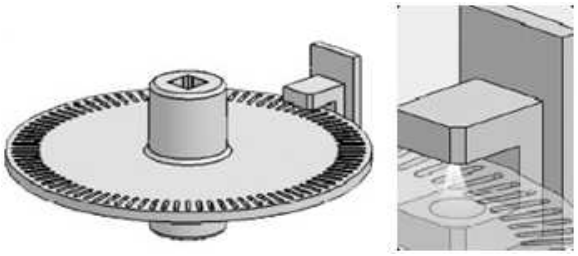


Figure 4: Shaft encoder.

Rollover

The pulses from an encoder are counted by a counter circuit with a finite number of bits, and, therefore, a maximum count. When the maximum or minimum count is reached, the counter ***rolls over***. For example, a 16-bit count interpreted as a two's complement number can count up to 32767, and then the next increment gives -32768. This is seen in figure 3 at t=48.5 seconds.

The SRV02 encoder counts 4096 counts/revolution, and the 16-bit counter in the Q2-USB can count a total of 65,536 counts, giving 16 revolutions between roll-over events.

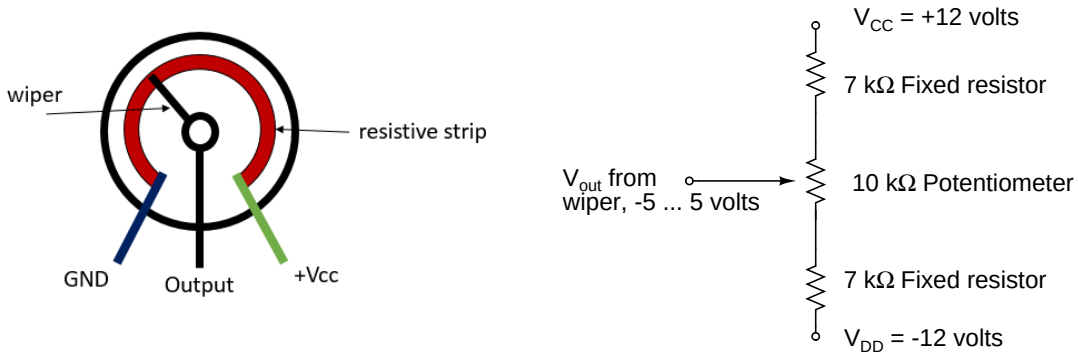
Incremental measurement

The encoder counter is set to zero when the USB cable of the Q2-USB is connected. So the arbitrary position of the output bar at that moment will correspond to an encoder value of zero. Thus the encoder shows ***DC offset***, which is the (possibly non-zero) sensor reading when the SRV02 output position equals zero.

There are several methods to provide the absolute registration for an incremental encoder, such as running a rotary link to a limit and setting the count value there, or including a second sensor that gives an ***absolute*** position signal (one that has the same value at the same position each time). For the SRV02, we can manually set the output bar to the zero position, and cycle the power on the Q2-USB by disconnecting and reconnecting the USB cable at the computer.

Note: Do not disconnect the cable at the Q2-USB end. Over 1 to 2 years, repeated connect/disconnect cycles will wear out the connector in the Q2-USB. It is better to wear out the connector in the computer.

1.3.1.2 Potentiometer A potentiometer is made with a resistive material and moving wiper, as illustrated in figure 5. As the wiper moves, the voltage changes. Clockwise rotation of the wiper of figure 5 would produce a voltage changing from 0 to +Vcc.



a) Potentiometer b) SRV02 circuit

Figure 5: Schematic diagram of a potentiometer, and SRV02 potentiometer circuit.

Potentiometers can be made ***continuously rotating***, which means the wiper can move many complete rotations without obstruction. With the potentiometer circuit of the SRV02 (figure 5.b), the sensed voltage ranges from -5.0 to +5.0 volts. This is seen for the potentiometer signal in figure 3. The continuously rotating wiper gives a voltage that rolls over once per revolution. Inspection of the roll-over events in figure 3 shows the rotation rate to be a bit more than one revolution per second.

Advantages: Advantages of a potentiometer as a position sensor are that it gives an absolute measurement (a given voltage always corresponds to the same position) and it is ***continuous*** (in principle as infinitesimal rotation can produce an infinitesimal change in voltage).

Disadvantages: Disadvantages are that the potentiometer has a ***deadband*** and DC offset, is subject to variations in calibration, and rolls over with each revolution. The rollover events

are apparent in figure 3 and can be used to get an estimate of the rotation rate (about 2π radians per second in figure 3).

The deadband can be seen by considering figure 5.a. When the wiper reaches the 6 o'clock position, it must leave the +Vcc end of the strip before connecting with the ground end of the strip. This deadband is not readily apparent in figure 3. But in section 2.3 you will explore sensor characteristics by setting motor voltage to zero and turning the output by hand. If you look for it, you can find the deadband.

The DC offset is due to the never-quite-perfect alignment of the output shaft gear and potentiometer gear. DC offset can be corrected by measuring the offset value during sensor calibration, and correcting for it in the Simulink model.

1.3.1.3 Tachometer The Faulhaber 2356S006S motor/tachometer unit used in the SRV02 comprises 3 elements:

- Motor
- Tachometer
- 14:1 planetary reduction gear

A tachometer is a DC generator, adapted to generate a voltage in proportion to rotational velocity.

Advantages: A tachometer directly indicates velocity, without sensor issues such as rollover or DC offset (the tachometer voltage will be zero at zero rotation rate). A quality tachometer also gives a very linear reading, from very low velocities to very high velocities.

Disadvantages: Both DC motors and generators are subject to a characteristic called ***magnetic cogging***, which arises because of the nonuniform shape of the magnetic field created by the poles of the armature (the rotating element) and stator (the stationary element). Cogging is not a manufacturing defect, it is an inherent aspect of the layout of these rotating machines. The upshot is a ripple in the tachometer output voltage. The Faulhaber 2251 tachometer is specified to have 7% of reading tachometer ripple (see specifications, appendix to Introduction).

Tachometer ripple is seen in figure 6, where the entire trace is 0.1 seconds. The approximate frequency can be estimated as rotation rate (1.4 revolutions per second) times the gear ratio (the tachometer turns at the motor speed) of 14:1, times the ripple frequency of 14 cycles/turn (the two values of 14 are a coincidence), giving 280 Hertz. This is above the Nyquist frequency for 500 Hz sampling, so aliasing is present in figure 6.

The upshot is that with this hardware, except a very low output speeds, tachometer ripple appears simply as noise. Figure 6 shows a significant level of noise. The signal in figure 6 has an RMS value of 0.58 [radians/sec], which approximately the specified 7% of the 9.0 [radians/sec] reading.

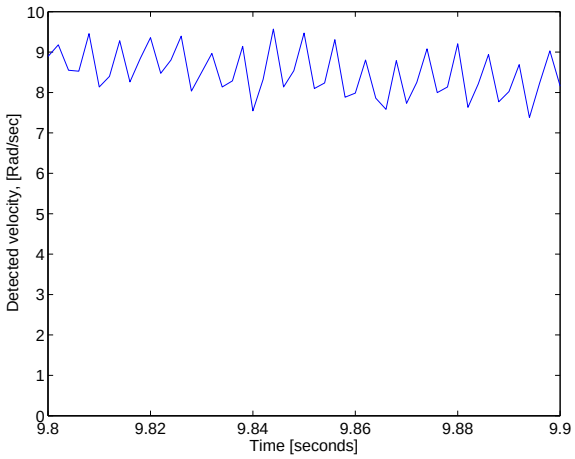


Figure 6: Zoomed-in portion of a velocity trace, showing tachometer ripple.

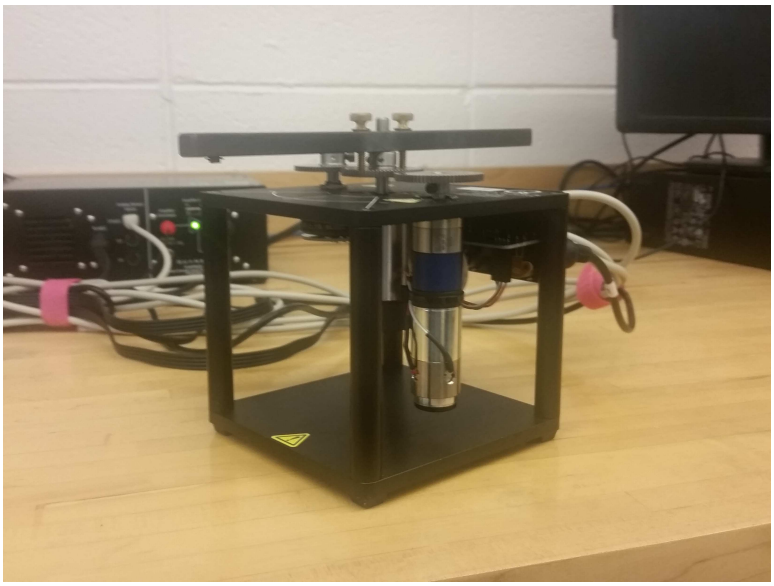


Figure 7: SRV02 Motor/Tachometer/Gear unit. The tachometer is the large lower portion, the motor is the middle, dark portion, and the reduction gear is the top light-colored portion.

The sensors used in the SRV02 - encoder, tachometer and potentiometer - are all common in industrial motion devices, such as robots. As we have seen, each of the sensors has strengths and weaknesses. Understanding the details and limits of the sensors is an important part of any feedback control application.

1.3.2 Models for the sensors

1.3.2.1 Encoder

The encoder indicates angle according to

$$p_e(t) = K_{e,0} + K_{enc}\, E(t) \qquad \text{[radians]} \tag{1}$$

where

$p_e(t)$ [radians] is the SRV02 output shaft angle (position), measured by the encoder at time t

$E(t)$ [counts] is the encoder count at time t

K_{enc} [radians/count] is the rate parameter of the encoder. For the SRV02 the rate parameter is

$$K_{enc} = \pm \frac{2\pi}{4096} \tag{2}$$

where the $+$ or $-$ comes from a possible direction reversal by the gears and will be determined below.

$K_{e,0}$ [radians] is the zero angle of the encoder, it is the angle of the SRV02 output shaft when the encoder counter indicates zero counts

1.3.2.2 Potentiometer

The potentiometer indicates angle according to

$$p_p(t) = K_{p,0} + K_{pot} V_{pot}(t) \qquad \text{[radians]} \tag{3}$$

where

$p_p(t)$ [radians] is the SRV02 output shaft angle, measured by the potentiometer at time t

$V_{pot}(t)$ [volts] is the potentiometer voltage at time t

K_{pot} [radians / volt] is the rate parameter of the potentiometer

$K_{p,0}$ [radians] is the zero angle of the potentiometer, it is the angle of the SRV02 output shaft when the potentiometer indicates zero volts.

1.3.2.3 Tachometer

The tachometer indicates angular velocity according to

$$v(t) = K_{tach} V_{tach}(t) \qquad \text{[radians/second]} \tag{4}$$

where

$v(t)$ [radians/second] is the SRV02 output shaft velocity, measured by the tachometer at time t

$V_{tach}(t)$ [volts] is the tachometer voltage reading at time t

K_{tach} [radians / second / volt] is the rate parameter of the tachometer

Thus calibrating the sensors, which you will be doing following exploring their basic characteristics, comprises:

- Encoder:
  - The magnitude of K_{enc} is known, for the US Digital model E2 encoder it is $\frac{2\pi}{4096}\left[\frac{\text{radians}}{\text{count}}\right]$
  - Determine or set K_{e0}
- Potentiometer:
  - Determine K_{pot}
  - Determine or set K_{p0}
- Tachometer:
  - Determine K_{tach}.

1.4 Prelab questions

- Answer these questions as the prelab write-up.
- Note questions marked "essay question" throughout the semester. For essay questions, write 1-2 complete sentences up to 1-2 complete paragraphs, as required to answer the question. Points will be deducted for incomplete sentences or illogical paragraphs.

Question 1. (Essay question, reply with 1 to 2 paragraphs) What is the relationship between Simulink, Quarc, the Real-Time Windows Target and the Q2-USB. Address these points:

- Which element(s) interact with the user
- Which element(s) provide real-time performance
- Which element(s) interface analog signals to the motor servo

Question 2. A program runs when the "Run" button is selected.

1. How is this program created ?
2. On what computer does the program run ?
3. If 8,000 measurements (updates) per second were required, would the system be adequate ?

Question 3. Data for the Digital to Analog Converter (DAC) of the Q2-USB is given in the apendices of the Introduction. With the data given above, what is the resolution of the Q2 DAC (that is, the minimum step in output voltage) ?

Question 4. Likewise, specifications for the Q2-USB Analog to Digital Converter (ADC) are given in the Introduction. What is the resolution of the Q2 ADC (that is, the change in input voltage that corresponds to a change in the digital value) ?

Question 5. How many turns can the sensors on the SRV02 record without rollover ? Address each sensor.

Question 6. Which sensor does not roll over ?

Question 7. Which sensor does not require measurements to determine the rate parameter ?

Example data, captured with the SRV02 while moving the output bar by hand, is seen in figure 8. The motion was first in the positive direction then, at about t=6.5 seconds, became negative-direction rotation.

Question 8. From figure 8, what are the signs of K_{enc}, K_{pot} and K_{tach} (that is, is each sensor rate-parameter a positive or negative value, considering the levels or slopes of the curves in the figure) ?

Question 9. From figure 8, what are the approximate numerical values of K_{enc}, K_{pot} and K_{tach} ?

Question 10. Matlab routine `DisplayStatistics.m` is provided to help analyze and take measurements from data. Read appendix 4.3 and answer these questions:

1. What is the minimum number of parameters for a call to `DisplayStatistics.m` ?
2. What is the maximum number of parameters for a call to `DisplayStatistics.m` ?
3. What tools does `DisplayStatistics.m` provide to help zoom in on a portion of a data set ?
4. If the data set of figure 8 were logged to the Matlab workspace though a Simulink scope, what would be the `DisplayStatistics.m` call, showing all needed parameters, to determine the average tachometer voltage on the interval from $t = 8$ to $t = 10$ seconds ?

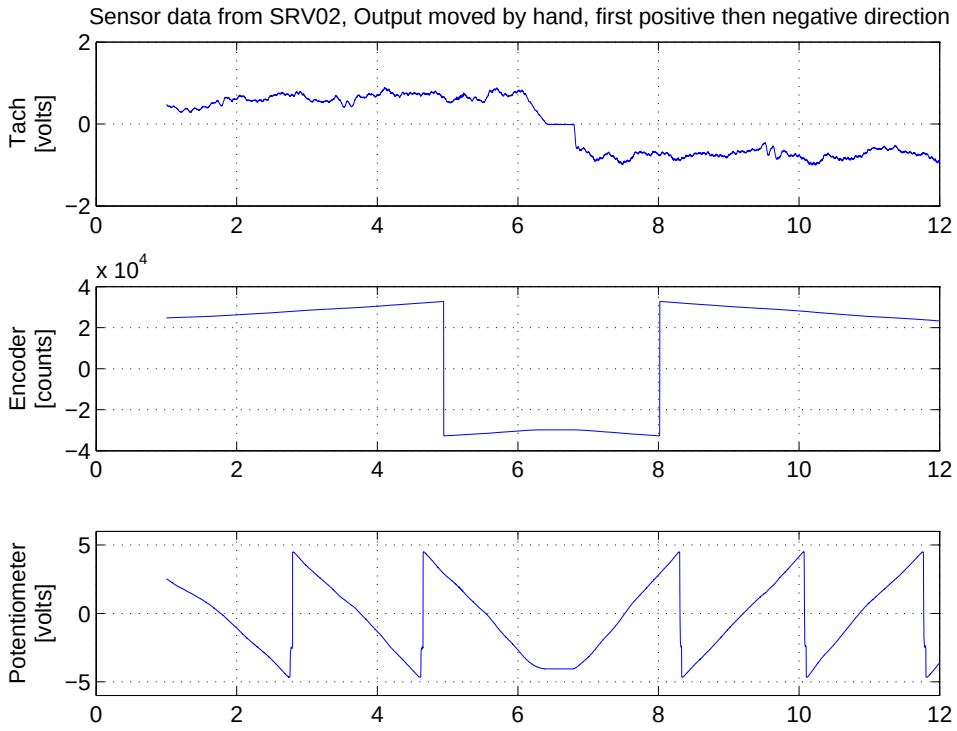


Figure 8: Example plots captured with manual rotation of SRV02.

2 In Lab

2.1 Using Quarc, Quanser's hardware in the loop tools

This section provides an introduction Quarc, Quanser's Hardware In the Loop (HIL) software tools. Quarc can be found in the Simulink blocks library, and full documentation can be found on Quanser's website. Here, the blocks used to operate the SRV02 are introduced.

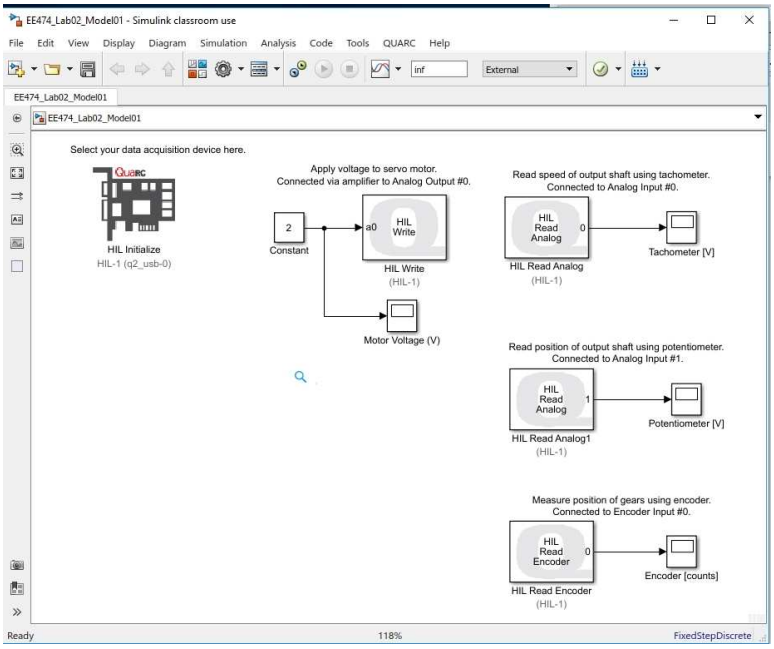


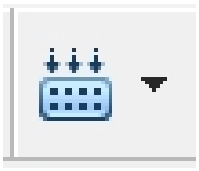
Figure 9: Simulink model `EE474_Lab02_Model01.mdl`.

2.1.1 Steps to build and run:

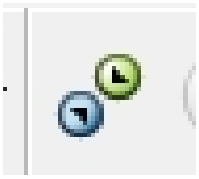
Three steps are essential to building and running a simulation model:

1. Activate "Build" 2. Activate Connect 3. Activate Run

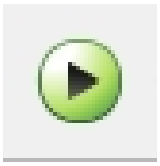
The twirler in the in the lower-right corner shows that Simulink is connected to the Target. The time incrementing, and signals progressing on the scopes, show that the system is running.



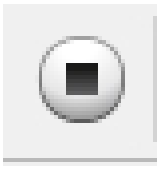
a) Build button



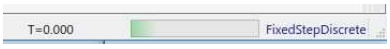
b) Connect button



c) Run button



d) Stop button



Twirler and time value

Figure 10: Build, Connect and Run buttons, and time and twirler from lower right corner.

2.1.2 HIL tools and an example Simulink model

Step 1. Install (if necessary) and open in Simulink model `EE474_Lab02_Model01.mdl`, seen in figure 9. This model has three groups of elements:

1. The Hardware In the Loop Initializer (this block initializes the Q2)
2. Blocks that generate the motor command voltage
3. Blocks that receive and record the the SRV02 sensors signals

Step 2. Open each block one after another and examine the parameters that can be set there.

- In the HIL Initializer block,
  - On the "Main" tab the Board type can be set.
  - On the Analog Inputs and Outputs tables, the voltage ranges are seen.
- In the motor command group,
  - Open the Constant block, and set the motor command value to zero.
  - The motor-command scope has logging to the Matlab workspace activated.
- In the sensor signal group,
  - Settings for the scope block are accessed with the gear icon on the scope window (open the scope, if the window is not open).
  - The three sensor scopes have logging to the Matlab workspace activated.

Step 3. Open the settings dialog for the simulation (the gear icon on the top banner of the model). The solver should be set to "Fixed-step" and "discrete," setting up the simulation to run at fixed increments of time. In "Advanced options" (or "Additional parameters") you will find "Fixed-step size." This is the ***sample period***, in seconds. It should be set to 0.002 seconds.

2.2 Cabling the SRV02

Setup of the SRV02 is described here. These details are also described in the Quanser document *Rotary Servo Base Unit - Quick Start Guide.pdf*.

A diagram of the SRV02 components and cabling is seen in figure 11.

- Quanser has configured the tools - the Quarc Simulink blocks, Q2-USB data acquisition board, VoltPAQ amplifier and SRV02 itself - to be quite flexible. A variety of combinations of Quanser equipment that can be assembled, and Quarc, the Q2-USB and the amplifier can be used in a variety of laboratory applications.

  An up-shot of the flexibility is some complexity with respect to the cabling. There are independent cables for the USB connection, each of the three sensors, and motor power. Quanser has used 6 different connector types, to help assure correct connections (for the most part, connectors only fit in their proper locations).

- Be sure VoltPAQ power is off (switch on rear) before installing or modifying cables.

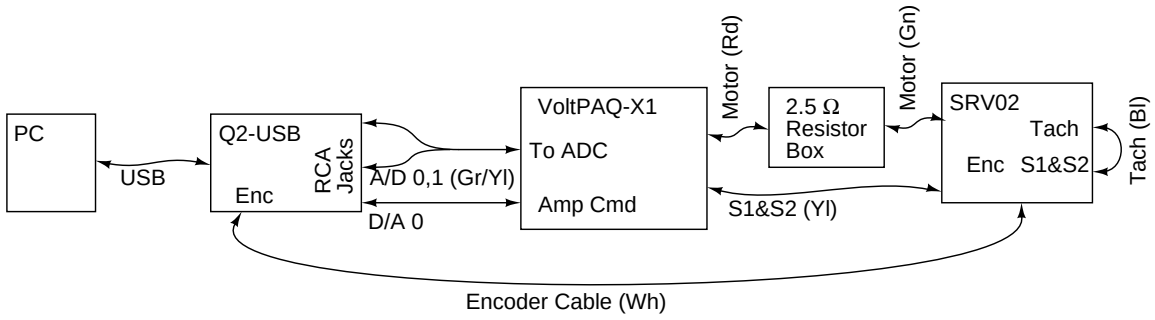


Figure 11: Diagram of SRV02 components and connections cabled in the standard configuration. Cable colors shown in parentheses.

- Computer -> Q2

  Step 4. Put in the USB cable

- Q2 -> VoltPAQ

  Step 5. Use the RCA to RCA cable to attach DAC channel 0 to VoltPAQ "Amplifier Command." This cable will carry the motor command voltage to the amplifier.

  (DAC = "Digital to Analog Converter". 8,000 times per second the Q2 DAC receives a numerical value between 0 and 4095 from the computer and converts it to a voltage between -10v and +10v).

  Cable the analog sensor channels with the 6-pin DIN to Dual-RCA cable. Connect the 6-pin DIN plug to the VoltPAQ at "To ADC", and connect the yellow and white RCA plugs to the Q2 (white to channel 0, for the tachometer, and yellow to channel 1, for the potentiometer).

  (ADC = "Analog to Digital Converter". The Q2 ADCs 0 and 1 receive voltages between -10v and +10v, which are converted to a numerical value between 0 and 4095. These are sent to the computer 8000 times per second.)

- SRV02 -> Q2

  Step 6. Use the 5-pin DIN to 5-pin DIN cable to connect the encoder output on the SRV02 to encoder channel 0 on the Q2.

- SRV02 -> VoltPAQ

  Step 7. Use the 6-pin DIN to 4-pin DIN cable to connect "Motor" on the SRV02 to "To Load" on the VoltPAQ. This cable carries the motor current.

  Use a 6-pin mini DIN to 6-pin mini DIN cable to connect an "S1&S2" jack on the SRV02 (either one) to the S1&S2 jack on the VoltPAQ. This cable carries the two analog sensor signals, for the tachometer and potentiometer.

- SRV02 -> SRV02

Step 8. Use a 6-pin mini DIN to 6-pin mini DIN cable to connect the tachometer output to the remaining "S1&S2" jack on the SRV02.

When finished, you should have four cables connected to the Q2, four cables connected to the VoltPAQ, and five cable connectors connected to the SRV02.

2.3 Exploring sensor characteristics

Start with a basic Simulink model, such as `EE474_Lab02_Model01.mdl` seen in figure 9, and set the motor voltage to zero.

Step 9. Run the model. With zero motor voltage the motor will not turn by itself, but can be turned by hand. Move the motor output by hand, and observe the three sensor signals in the corresponding scopes. Note questions 15 and beyond, below. Here are some suggested things to explore

- For each of the sensors, what is the value when the bar is in the zero position (or zero velocity)
- Move the bar in the positive direction (right-hand rule, with your right thumb pointing up along the line of the output bar shaft, the positive direction is the direction your right fingers curl),
  - Does the tach indicate a positive or negative voltage (giving the sign of K_{tach})
  - Do the encoder and potentiometer outputs progress in the positive or negative directions (giving signs of K_{enc} and K_{pot})
- Apply a positive voltage. (Open the dialog of the motor voltage block in the simulink model and change the value, for example, to 2 volts, and select Apply or OK. This can be done while the model is running.)

  Does the bar move in the positive direction (is the DC gain positive or negative) ?

  Set the output voltage back to zero.
- Using the scopes, starting with the output bar in the position that gives zero encoder counts, how many complete revolutions can be made before the encoder count rolls over ?
- Similarly using the scopes, starting with the output position that gives zero potentiometer volts, how much rotation of the output is possible before rollover ? Is the maximum rotation before rollover the same rotating in the positive and negative directions ?
- Setup a steady motion with an applied voltage, and log 10 or more seconds of data to the Matlab workspace. Record this data withthe 'save' command for reference later.

Note: Model `EE474_Lab02_Model01.mdl` is setup to log data to the Matlab workspace (see "Logging" in the scope settings). A new log is started each time you select "Run". The data are transferred to the Matlab workspace when you select "Stop."

Routine `DisplayStatistics.m` is provided to graph data captured to the workspace, see section 4.3.

2.4 Calibrating the sensors

In this process, we will use the fact that the rate parameter of the encoder is known to calibrate the rate parameters of the pot and tach. K_{tach} , K_{pot}, K_{e0} and K_{p0} must be determined or zeroed.

- Set the motor voltage to a value that gives 1-2 revolutions per second in the positive direction
- With the motor turning at a steady speed (steady voltage, and after the startup transient) record about 10 seconds of data

In Matlab

- Determining K_{tach}
  - Select an interval of the data with steady motion and without a rollover of the encoder, for example from 44 to 48 seconds in figure 3
  - Using the encoder, determine the total amount of rotation
  - Using the time signal, determine the total time interval
  - Determine the average rotation rate
  - Determine K_{tach}

$$K_{tach} = \frac{\text{Average rotation rate}}{\text{Average voltage}} \qquad \left[\frac{\text{radians / second}}{\text{volt}}\right] \tag{5}$$

    Pay attention to the signs of the total rotation, average rotation rate and K_{tach} .

- Determining K_{pot}
  - Select an interval of the data with steady motion and without a rollover of either the encoder or pot, for example from 47.0 to 47.5 seconds in figure 3 (zooming in to the figure will be necessary to get accurate endpoint times, be sure to include only steady motion shown by the pot, excluding rollover effects)
  - Using the encoder, determine the total amount of rotation (in radians)
  - Using the pot signal, determine the change in voltage of the pot
  - Determine K_{pot}

$$K_{pot} = \frac{\text{Total rotation}}{\text{Total change in voltage}} \qquad \left[\frac{\text{radians}}{\text{volt}}\right] \tag{6}$$

    Of course, pay attention to the signs.

- Determining K_{p0}

  - Set the applied voltage to zero
  - Rotate the output bar by hand to the zero position

    The output bar can be accurately placed in the the zero position by sighting vertically through a hole in the output bar and aligning with the center marking on the SRV02 frame.
  - Run the simulation for a few seconds
  - In Matlab, average at least 100 samples of voltage (to reduce the effect of electrical noise), and determine the average pot voltage at the zero position. Parameter K_{p0} is the position at zero volts, given by:

$$K_{p0} = -K_{pot} \text{ (Average voltage at zero position)} \qquad [\text{radians}] \tag{7}$$

- Setting K_{e0}

  K_{e0} is the one calibration value that can be set, by zeroing the encoder counter.
  - Set the output bar accurately to the zero position
  - Zero the encoder counter

    One method to zero the encoder counter is to unplug the computer-end of the USB cable, count to 10 and re-plug the USB cable. On power-up, the encoder counters are reset.
- As is true with industrial systems, the sensor parameters will vary between units. Record the unit number of your SRV02, as well as the Q2-USB and VoltPAQ. If you are using a different unit in a future lab, you will need to calibrate that unit.

2.5 Consider the write-up

- While in lab, consider the questions in the write-up, and collect any data or observations needed.

2.6 Consider the principal values

For this lab, the principal values are

K_{enc} , K_{tach} , K_{pot}, and K_{p0}

For full credit, you must determine these values to within $\pm 5\%$. This can involve some careful consideration of the data. For example, to determine K_{pot}, you measure a change in pot voltage on an interval ("Total change in voltage"), corresponding to the encoder rotation measurement over the same interval ("Total rotation"). If the interval selected is too short, quantization and sensor noise will introduce errors. If the interval selected includes a portion of the pot deadband, the measurements will be off.

Tips (valid for all 7 labs):

1. Save your data, so you can reload it and reanalyze it when questions arise. This can be done with

```
>> save Lab02_DataSet01_a
...
>> save Lab02_DataSet01_b
...
```

2. Save at least 3 repetitions of the data (that is, repeat the measurement at least 3 times).

   Three is kind of a magic number for data collections, because if there is an outlier, in many cases two measurements will agree and the outlier can be determined. With one repetition of the measurement there is no way to know an outlier occurred, with two there is no way to know which is the outlier.

   With three repetitions you can average the result. Averaging three measurements will cut the contribution of random noise by the square root of 3.

3. Where straight forward to do, write a Matlab script to do the analysis. Then you can run your analysis of the several data sets with:

```
>> Load Lab02_DataSet01_a
>> MyScript
>> Load Lab02_DataSet01_b
>> MyScript
...
```

3 Write-up

- Write a report, as described in the *"Style Guide for EE 474 Postlab Write-up"*. Include the Principal Values and Graphs and answers to the questions below.

3.1 Principal Values

- For lab part 2 the principal values are

  K_{enc} , K_{tach} , K_{pot}, and K_{p0}

  There are no Principal Graphs.

3.2 Questions

- Questions, section 2.1

  Question 11. Considering the HIL initializer,

1. What is the board type ?
2. What are the minimum and maximum voltages for the analog inputs ? What are the minimum and maximum voltages for the analog outputs ?

Question 12. Considering the Write block

1. What is the time span for the Motor Voltage scope ?
2. What is the analog channel for the HIL write block ?

Question 13. Considering the Read Analog blocks

1. In the HIL Read Analog block for the tachometer, which channel is read ?
2. In the HIL Read Analog block for the potentiometer, which channel is read ?
3. What is the Matlab variable name for logging for the Tachometer scope ?

- Questions, section 2.2

Question 14. How many cables are involved in setting up the complete SRV02 system ?

- Questions, section 2.4

Question 15. (Essay question) Write a paragraph or two about the strengths and weaknesses of each sensor (that is, 3 to 6 paragraphs total). Include at least one figure for each sensor, showing recorded data, to illustrate one or more of the items you describe about that sensor.

Question 16. Calibration of the SRV02

1. Draw a block diagram of the configuration used to calibrate the sensors, including the ID number of each piece of equipment.
2. What are the calibration coefficients of your SRV02 ?

Question 17. If the output shaft is rotated by 1 radian, and without rollover,

1. What will be the change in the encoder count ?
2. What will be the change in pot voltage ?

Question 18. What is the negative limit of the encoder value, before rollover ? What is the positive limit ? (If your data set does not include rollover, record a new data set. Use `DisplayStatistics.m` to display data quickly.)

Question 19. Starting with the output bar in the position that gives zero encoder counts, how many complete revolutions can be made before the encoder count rolls over ?

Question 20. For the potentiometer, how much rotation of the output is possible before rollover ? Is the maximum rotation before rollover the same rotating in the positive and negative directions ?

Note: Questions 21 and 22 ask about characteristics of the sensed signal. The units for resolution of a sensor and ripple are the units of the sensor. That is, radians and radians/second for the SRV02. Appendix "Precision, accuracy and resolution of a sensor" of the EE 474 Lab Introduction has additional information.

Question 21. The Analog to Digital Converter (ADC) converts a voltage to an integer. The ADCs in the Q2-USB have 12-bit resolution, which is to say they can distinguish 4096 voltage levels between -10.0 volts and + 10.0 volts. This gives a quantization (maximum resolution) of 20v / 4096 = 0.0049 volts.

1. Considering the calibration of the potentiometer, what is the resolution of position measurement (in radians) using the potentiometer ?
2. How does this compare with the resolution (in radians) of the encoder ?
3. What is the resolution of tachometer (in radians/second) ?

Question 22. When the SRV02 is rotating 10 [radians/second], what is the approximate amplitude of the tachometer ripple ?

4 Appendices

4.1 Simulink solver setup

To work successfully with Quarc, the needed configuration is quite specific, and it is recommend that you copy a known-working Simulink model to inherit all attributes of a working configuration. Here are some of the required parameters

- In Configuration Parameters (gear on main model page)
  - Type: Fixed step
  - Solver : ode1 (Euler)

  Fixed-step size (fundamental sample time): 0.002

4.2 Scope setup for logging to Matlab workspace

Elements of scopes are controlled in two different places

1. The scopes are triggered, so that they all start together. They can be triggered in a variety of ways, like a standard oscilloscope. To setup triggering, select "Code" at the top of the main Simulink window, the select "External Mode Control Panel," the select Signal & Triggering

   ```
   Code -> External Mode Control Panel -> Signal & Triggering
   ```

   In Signal & Triggering, set

   Source: manual

   Mode: normal

   Duration: 500000

   The duration, in particular, sets the interval before signal logging is re-triggered. In EE 474, we run the SRV02 at 500 samples/second, and so the indicated value of 500000 will give a re-triggering event in 1000 seconds. We do not record any events 1000 seconds long, and so setting to 500000 is equivalent to turning off retriggering.

2. The settings dialog of the individual scope

   (a) On the "Logging" panel

   Limit data points to last: `nPoints`

   Select, and fill in variable nPoints. **Note:** variable `nPoints` must be set in the Matlab workspace. Using a variable (as opposed to typing in a numerical value) will permit the data length of all scopes to be changed together.

Decimation:	Deselect
Log data to workspace:	Select
Variable name:	<Set desired variable name, such as "TachometerOutput">
Save format:	Structure With Time

(b) On the "Main" panel, the number of inputs is set. The separate inputs appear as columns in the array

```
<ScopeName>.signal.values .
```

(c) Other parameters on Main, Time and Display control how the scope is displayed. Typical settings of some fields are

Axis scaling:	Manual or Auto
Time span:	10
Times span overrun action:	Scroll

4.3 Matlab routine `DisplayStatistics.m`

In this and future labs you will be evaluating quantitative characteristics of the step response and steady rotations. It will be handy to have a routine to systematically process recorded data and display several statistical measures. Routine `DisplayStatistics.m` is provided to analyze and filter data. A basic call to `DisplayStatistics.m` has the form:

Call 01: `DisplayStatistics(TachometerOutput.time, TachometerOutput.signals.values)`

where `TachometerOutput` is produced by the Simulink scope, `.time` is a column vector of times and `.signals.values` is a column vector of data. DisplayStatistics.m has these functions:

1. A graph of the total data set
2. The user can specify t_1 and t_2 , and a blow-up graph of the interval t_1 ... t_2 is drawn.
3. Displaying statistics

   When DisplayStatistics.m is run, these values are printed to the screen:

   - Mean
   - RMS
   - Max
   - Standard deviation
   - Min

   If an interval t_1 ... t_2 is specified, the statistics reflect data on the range t_1 ... t_2, otherwise the statistics are for the full data set.
4. Setting the figure number, sometimes it is useful to have plots of several data sets up at once
5. Low-pass filtering,

   If an optional cut-off frequency is set, the data are filtered using a 3rd order Butterworth digital low-pass filter. See figure 14 below. Matlab's `filtfilt()` command is used, to minimize phase shift.

6. Return values

   Optionally, the statistical values are returned, for use in further processing.

For example, Call 01 above applied to data set Lab04_ExampleData01, gives figure 12, and Matlab prints to the screen:

```
Display Statistics report:
  Processed from index    1 to index 9577
  Mean   :   17.2166
  StdDev :    2.9847
  RMS    :   17.4734
  Min    :   -0.0488
  Max    :   21.1556
```

The report indicates the range of evaluation and the five statistics.

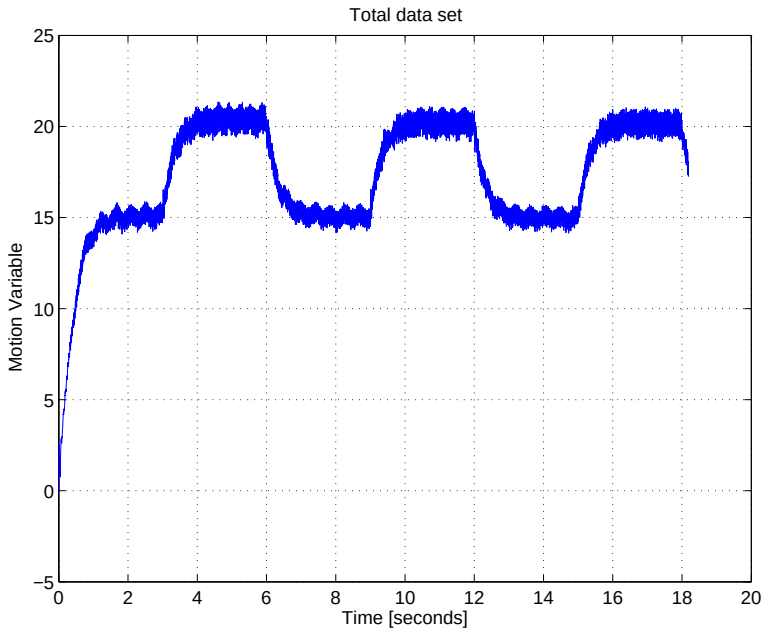


Figure 12: Data set Lab04_ExampleData01, full plot.

Typically, we are interested in a subset of the complete data set. For example, we might examine the full data set of figure 12 and determine that the region of interest is the 20 [rad/sec] interval from $t = 3.0$ to $t = 6.0$ seconds. Then make the call

Call 02: `DisplayStatistics(Times, DataValues, 3, 6);`

where `Times` is the vector of time values, and `DataValues` is the vector of data values. With Call 02, DisplayStatistics.m plots two plots; the first plot is the full plot, the second plot is seen in figure 13. And Call 02 results in this statistics report printed to the screen:

```
Display Statistics report:
  Processed from index 1501 to index 3001
  Mean   :   19.8199
  StdDev :    1.0856
```

```
RMS     :   19.8495
Min     :   15.1242
Max     :   21.1556
```

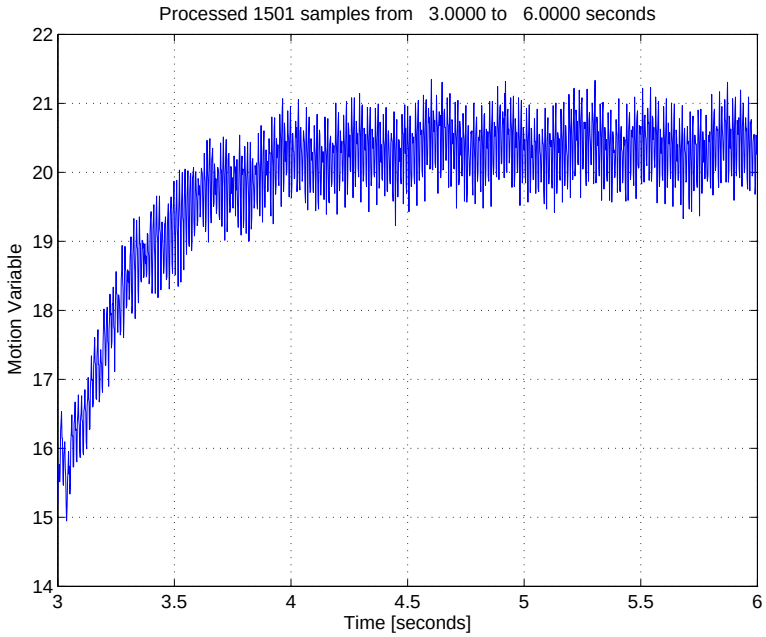


Figure 13: Data set Lab04_ExampleData01, blow-up of range 3.0 to 6.0 seconds. The strong noise signal comes from tachometer cogging. It has very high frequency and can be filtered by a 20 Hz filter.

The fifth parameter is the figure number to which to plot, and the sixth is the filter cut-off frequency. Parameters 3 to 6 are optional, and are seen in Call 03.

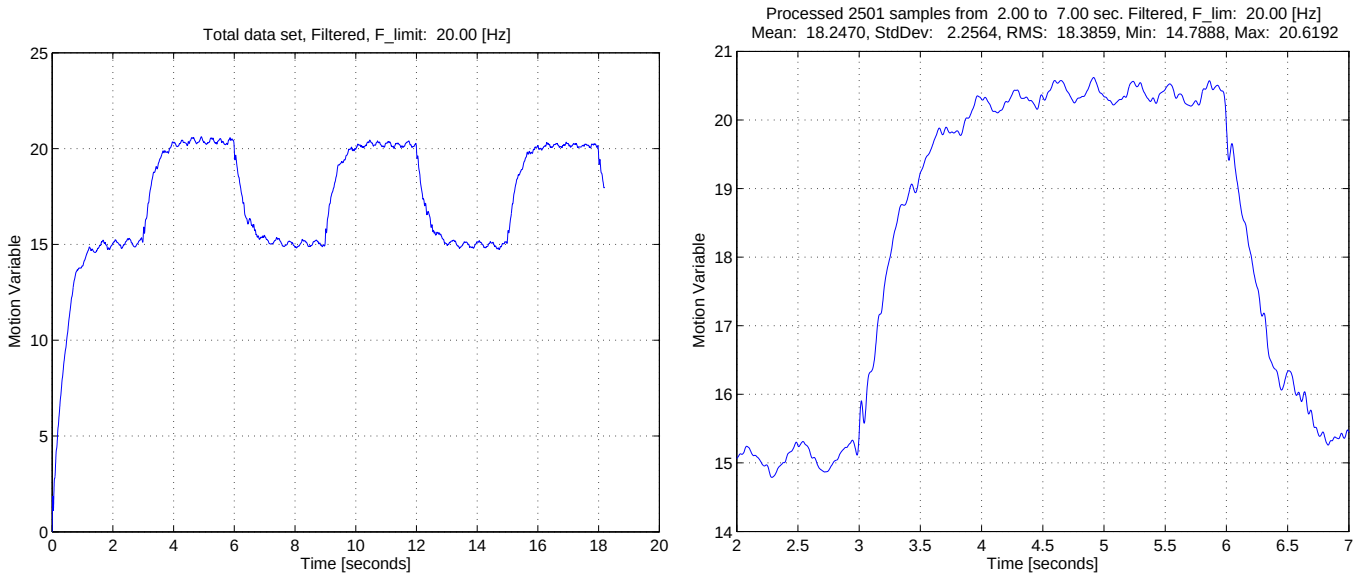
Call 03: `DisplayStatistics(Times, DataValues, 2, 7, 5, 20);`

The resulting plots are seen in figure 14, the tachometer-cogging noise signal is removed. A once-per-revolution signal is seen in the velocity. This is a true velocity signal that arises with once-per-revolution variations in gear friction. The amplitude of the velocity variations can be measured with Call 04.

Call 04: `DisplayStatistics(Times, DataValues, 4.5, 5.9, 5, 20);`

Call 04 produces the plot of figure 15, and statistics report below. If we are interested to determine the amplitude of the frictional disturbance response, we can determine peak-to-peak as Max-Min, giving 0.4167 [radians/second]. Alternatively, we could evaluate the RMS value. The RMS statistic itself includes the DC component, which is unwanted for looking specifically at the periodic signal. RMS and standard deviation are closely related, except that the mean is removed for standard deviation, so with a small adjustment standard deviation can be taken as the RMS of the signal with the mean removed,

$$\mathrm{RMS}(y) = \sqrt{\frac{n-1}{n}}\,\mathrm{std}\,(y) \tag{8}$$



(a) (b)

Figure 14: Plots from Call 03. The cyclic signal seen in figure 14(b) arises with a once-per-revolution friction signal, arising with slight eccentricity in the gears.

Using Eqn (8), we find RMS $(y) = \sqrt{\frac{700}{701}}\, 0.1110 = 0.1109$ [radians/second] as the RMS magnitude of the frictional disturbance in figure 15. The small adjustment, 0.9993 in this case, is often within the experimental uncertainty and can be neglected.

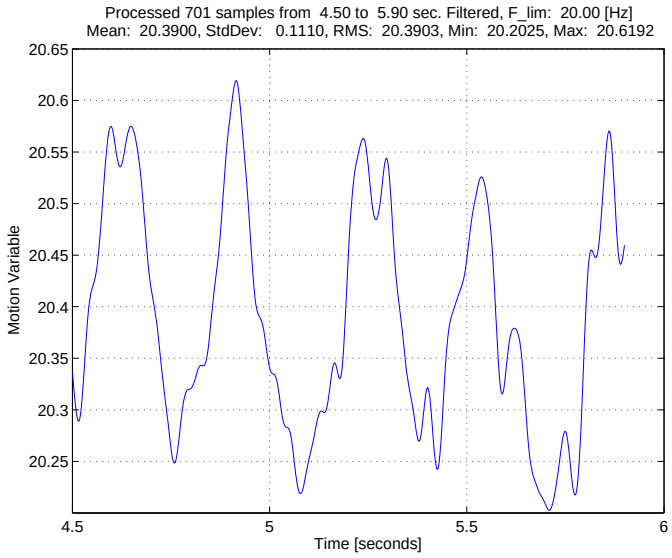


Figure 15: Blow-up of the filtered data of figure 14, with the range 4.5 to 5.9 seconds. These times are chosen, $t_1 = 4.5$ seconds to avoid the rising transient seen in figure 14, and $t_2 = 5.9$ seconds to avoid the first sample of the downward transient.

```
Display Statistics report:
  Processed from index 2251 to index 2951
```

```
Filtering is on, F_limit:  20.00
Mean   :   20.3900
StdDev :    0.1110
RMS    :   20.3903
Min    :   20.2025
Max    :   20.6192
```

Call 05:
```
[Mean, StdDev, RMS, Min, Max, FilteredData] = ...
      DisplayStatistics(Times, DataValues, 3, 6);
```

Call 05 calls DisplayStatistics with an interval of interest from $t = 3.0$ to $t = 6.0$ seconds, and returns parameters `Mean, StdDev, RMS, Min, Max` and the `FilteredData` to the Matlab workspace, for further processing.